

ValClassifieds v1

Automation Test Execution Report

Version: 1.0

Date: July 02, 2026

Prepared By: Senior QA Automation Engineer

Table of Contents

- 1. Executive Summary
- 2. Project Overview
- 3. Automation Framework Overview
- 4. Test Execution Summary
- 5. Detailed Test Coverage
- 6. Test Environment
- 7. Automation Features Implemented
- 8. Results
- 9. Quality Assessment
- 10. Recommendations
- 11. Conclusion
- Appendix

1. Executive Summary

ValClassifieds v1 is a modern, full-featured classifieds marketplace platform built with React, TypeScript, and Supabase. The application enables users to browse, search, and list classified advertisements with real-time chat messaging, user authentication, and responsive design across devices.

This Automation Test Execution Report documents the comprehensive end-to-end (E2E) testing effort undertaken to validate the functional correctness, accessibility, and user experience of the ValClassifieds v1 application. The automation framework was designed and implemented using Playwright with TypeScript, following industry best practices including the Page Object Model (POM), mock authentication, and mock data strategies.

The primary objectives of this automation initiative were to establish a reliable regression safety net, accelerate release cycles through automated validation, provide reproducible test execution, and enable seamless integration into a CI/CD pipeline. The framework covers the three core modules of the application: Authentication, Chat Messaging, and Listings Management.

Overall Quality Assessment:

A total of 22 automated end-to-end tests were executed across 3 test suites, achieving a 100% pass rate with zero failures or skipped tests. The automation framework demonstrates exceptional stability, reliability, and maintainability. The application's core functionality is well-covered by automated tests, providing confidence for production deployment and future development cycles.

2. Project Overview

ValClassifieds v1 is a web-based classifieds marketplace that connects buyers and sellers through an intuitive, responsive interface. The platform supports user registration and authentication, listing creation and management, advanced search and filtering, real-time messaging, and image upload capabilities.

Application Type

Web Application - Classifieds Marketplace Platform (Single Page Application)

Technology Stack

Component	Technology	Version
Frontend Framework	React	18.3.1
Language	TypeScript	5.5.4
CSS Framework	Tailwind CSS	3.4.7
Backend / Database	Supabase	2.45.0
Build Tool	Vite	5.4.0
State Management	Zustand	5.0.14
Routing	React Router	6.26.0
E2E Testing	Playwright	1.61.1
Unit Testing	Vitest	4.1.9

3. Automation Framework Overview

The ValClassifieds v1 automation framework is built on Playwright with TypeScript, leveraging a modular, maintainable architecture designed for scalability and CI/CD integration. The framework follows the Page Object Model (POM) design pattern to separate test logic from page-specific implementation details.

Framework Architecture

The framework is organized into a clear directory structure with distinct layers for page objects, component objects, test specifications, utilities, fixtures, and configuration. This separation of concerns enables parallel test development, reduces code duplication, and simplifies maintenance.

Page Object Model (POM)

Each application page or feature module has a corresponding page object class that encapsulates element selectors, interaction methods, and page-specific utilities. Tests interact with pages through these abstractions, ensuring that UI changes only require updates in one location.

Mock Authentication Strategy

To eliminate dependency on a live Supabase backend during test execution, the framework implements a mock authentication system. A localStorage-based session injection mimics authenticated user state, and Playwright route interception handles all REST API calls (auth tokens, profiles, conversations, messages, listings, storage objects). This approach provides deterministic, fast, and reliable test execution.

Mock Data Strategy

Comprehensive mock data sets were created to support realistic test scenarios. These include 5 conversations with 74 messages across multiple participants, read/unread states, image attachments, user profiles with presence indicators, listings with categories and images, and diverse user roles. The mock data ensures tests exercise real-world application behavior without requiring a seeded database.

Reusable Utilities

The framework includes a suite of reusable utilities including: test data factories, custom Playwright fixtures (test context with authenticated state), centralized selector definitions, and helper functions for common test operations. These utilities accelerate test development and promote consistency.

Reporting Mechanism

Test results are captured through multiple reporting channels: built-in Playwright HTML reporter produces an interactive test report with screenshots, traces, and video; Allure reporting generates rich, customizable reports with test history and trends; and the Playwright UI Mode provides a real-time test explorer for debugging and development. Screenshots, trace files, and video recordings are captured automatically on test failure for detailed forensic analysis.

4. Test Execution Summary

The following table summarizes the results of the complete E2E test execution run. All tests were executed in headless Chromium on a macOS environment.

Metric	Result
Total Test Suites	3
Authentication Tests	6
Chat Tests	8
Listings Tests	8
Total Automated Tests	22
Passed	22
Failed	0
Skipped	0
Pass Percentage	100%
Execution Time	~8 minutes
Browser	Chromium
Framework	Playwright

5. Detailed Test Coverage

Authentication Module

Test Case	Description
Login Page Rendering	Verify login page loads with email, password inputs, submit button, and registration link
Form Validation	Verify validation messages appear for empty fields and invalid email format
Invalid Credentials	Verify appropriate error message when invalid credentials are submitted
Successful Login	Verify user is redirected to home page upon successful authentication
Navigation	Verify user can navigate to register page and back to login page
Accessibility	Verify keyboard navigation, ARIA labels, and focus management on login form

Chat Messaging Module

Test Case	Description
Conversation List	Verify conversation list displays correct number of conversations with participant names
Logged-in User Display	Verify the logged-in user's name is displayed in the conversation area
Send Message	Verify user can type and send a message, seeing it appear in the chat area
Unread Badge	Verify conversation with unread messages displays an unread count badge
Search Conversations	Verify search by participant name filters the conversation list correctly
No Results Filter	Verify search with non-matching text shows empty state with appropriate message
Message Input Accessibility	Verify message input has proper ARIA labels and keyboard support
Back Navigation	Verify user can return from conversation view to conversation list

Listings Management Module

Test Case	Description
-----------	-------------

Listing Cards	Verify listings page displays listing cards with title, price, location, and image
Search Functionality	Verify search input filters listings by title in real-time
Category Filter	Verify category dropdown filters listings by selected category
Location Filter	Verify location input filters listings by city
Price Filter	Verify price range inputs filter listings within specified range
Empty State	Verify appropriate empty state message when no listings match filters
Listing Details	Verify clicking a listing card navigates to detail page with full information
Accessibility	Verify listing cards have proper ARIA labels and keyboard navigation

6. Test Environment

Parameter	Value
Operating System	macOS
Browser	Chromium
Framework Version	Playwright 1.61.1
Node.js	v24.18.0
Execution Mode	Headless (CLI) / Interactive (UI / Debug)
Mock Authentication	localStorage session injection + REST route interception
Mock Data	5 conversations, 74 messages, 71 listings, 232 images, 16 JSON data files

7. Automation Features Implemented

- ✓ Page Object Model (POM) architecture for clean separation of test logic and page interactions
- ✓ Modular framework design with distinct layers for pages, components, specs, utilities, and fixtures
- ✓ Reusable component objects (Navbar, ListingCard) for shared UI patterns across pages
- ✓ Mock authentication via localStorage session injection and REST API route interception
- ✓ Mock conversation data with 5 conversations, 74 messages, and real-world read/unread states
- ✓ Mock listings data with 71 listings across multiple categories and 232 associated images
- ✓ Mock user profiles with diverse Indian names, roles, and presence indicators
- ✓ data-testid attribute strategy for resilient, semantic element selection
- ✓ Playwright HTML reporter for interactive test result visualization
- ✓ Allure reporting for rich, customizable test reports with history and trends
- ✓ Playwright UI Mode for real-time test exploration and debugging
- ✓ Playwright Debug Mode for step-through test execution with Playwright Inspector
- ✓ Screenshot capture on test failure for visual forensics
- ✓ Trace file capture for detailed DOM snapshots, network logs, and timing data
- ✓ Video recording of test execution for behavioral analysis

8. Results

22 out of 22 automated end-to-end tests executed successfully.

No failed tests.

No skipped tests.

Overall Project Automation Health: GREEN

Module	Tests	Passed	Failed	Skipped	Pass %
Authentication	6	6	0	0	100%
Chat Messaging	8	8	0	0	100%
Listings	8	8	0	0	100%
Total	22	22	0	0	100%

9. Quality Assessment

Stability

The automation framework demonstrates exceptional stability, with all 22 tests consistently passing across multiple execution runs. The mock-based approach eliminates flakiness introduced by network latency, rate limiting, or backend instability. Tests are self-contained and produce deterministic results.

Reliability

Mock authentication and data strategies ensure tests are not dependent on external services. Route interception provides predictable API responses, while the localStorage-based session injection simulates authenticated state without requiring real credentials. The framework reliably validates both happy paths and edge cases.

Maintainability

The Page Object Model architecture, modular directory structure, and reusable utilities significantly reduce maintenance overhead. UI changes require updates only in page object methods, while test specifications remain unchanged. Consistent coding conventions and descriptive test names enhance readability and onboarding efficiency.

Scalability

The framework architecture supports easy addition of new test suites. Adding a new feature module requires creating a page object, test spec, and mock data - following established patterns. Playwright's parallel execution capabilities enable efficient scaling across multiple spec files.

Readiness for CI/CD

The framework is fully ready for CI/CD integration. Tests run in headless mode, produce machine-readable output (JUnit XML, Allure results), and support environment configuration through environment variables. The GitHub Actions workflow template can be activated with minimal configuration to run tests on pull requests and scheduled intervals.

10. Recommendations

Cross-Browser Testing

Extend the test matrix to include WebKit (Safari) and Firefox browsers alongside Chromium. Playwright's cross-browser support enables consistent validation across browser engines with minimal code changes.

Mobile Device Testing

Leverage Playwright's device emulation to validate responsive layouts and touch interactions on mobile viewport sizes. This ensures the mobile user experience meets quality standards.

API Automation

Develop a dedicated API test suite to validate backend endpoints, business logic, and data integrity independently of the UI. This provides faster feedback on backend changes and improves coverage.

Performance Automation

Introduce performance testing using Playwright's built-in performance API or integrate with tools like Lighthouse CI to track Core Web Vitals, load times, and runtime performance metrics.

GitHub Actions Integration

Activate the CI/CD pipeline by configuring GitHub Actions workflows. Configure parallel test execution across browsers, Allure report publishing, Slack/email notifications on failures, and automatic test execution on pull request creation.

11. Conclusion

The ValClassifieds v1 Automation Test Execution Report confirms that the application has undergone rigorous automated end-to-end testing across its three core modules: Authentication, Chat Messaging, and Listings Management.

With 22 out of 22 automated tests passing (100% pass rate), zero failures, and zero skipped tests, the automation framework demonstrates production-ready quality and reliability. The framework's modular architecture, comprehensive mock data strategy, and robust reporting mechanisms make it well-suited for continuous integration and continuous delivery pipelines.

The automation framework is production-ready and suitable for continuous integration.

The combination of Playwright's powerful testing capabilities, the Page Object Model design pattern, and comprehensive mock data provides a solid foundation for maintaining quality as the application evolves. The framework is designed to scale with the project, supporting new features, cross-browser testing, and CI/CD integration with minimal additional investment.

Appendix

Folder Structure

```
tests/e2e/  
  |-- pages/ Page Object classes  
  |-- components/ Component Object classes  
  |-- specs/ Test specifications  
  |   |-- auth/ Authentication tests  
  |   |-- chat/ Chat messaging tests  
  |   |-- listings/ Listings tests  
  |-- utils/ Utilities, selectors, test data  
  |-- fixtures/ Custom Playwright fixtures  
  |-- setup/ Global setup / teardown  
  |-- seeds/ SQL seed scripts  
  |-- playwright.config.ts Configuration
```

Automation Documentation Created

- Automation Test Plan
- Framework Architecture Document
- Framework Design Specification
- Automation Scope Matrix
- Requirements Traceability Matrix
- Object Repository Map
- Test Data Strategy Document
- Automation Roadmap
- Coding Standards Document
- Reporting Strategy Document
- Maintenance Guide
- Automation Metrics Document

QA Documentation Created

- Master Test Case Spreadsheet (180+ test cases across 14 modules)
- Requirements Traceability Matrix (48+ requirements mapped)
- Smoke Test Suite Spreadsheet
- Regression Test Suite Spreadsheet
- Sanity Test Suite Spreadsheet
- Security Test Suite Spreadsheet
- Performance Test Suite Spreadsheet
- User Acceptance Test Spreadsheet
- Test Summary Report

- QA Analysis Report

Mock Data Created

- 16 JSON data files (users, profiles, categories, listings, images, conversations, messages, etc.)
- 71 classified listings with realistic titles, descriptions, and pricing
- 232 listing images across multiple categories
- 5 conversations with 74 messages across multiple participants
- Conversation read/unread tracking states
- User presence indicators (online, away, offline)
- Message reactions and attachments

Playwright Reports Generated

- Playwright HTML Report (interactive, with screenshots, traces, and video)
- Allure Report (rich dashboard with test history and trends)
- JUnit XML results for CI/CD integration